Cape Peninsula University of Technology

creating futures

Department of Electrical Electronics and Computer Engineering

SDN260S – Software Design II

Project Brief

INSTRUCTIONS

1. Read the group project guideline carefully and ensure you understand the requirements of your chosen project topic.
2. Install **Apache NetBeans** or **MS Visual Studio Code** for implementation. Refer to the Blackboard resource folder for setup instructions and recommended tools.
3. Each group is expected to work collaboratively on the project, applying concepts learned in class (system design, modularity, OOP, algorithms, testing, and documentation).
4. Submit the following deliverables as your Group Project Assignment on Blackboard:
    - A single PDF report containing:
        - Problem definition and requirements. System design and architecture (with diagrams). Implementation details (class descriptions, algorithms, advanced features). Testing and evaluation results. Reflection on graduate attributes (GA2 & GA7). Team contribution breakdown
    - The complete project source code placed in a zipped file labeled: "GroupNumber_ProjectTitle"
5. Late submissions will result in a deduction of marks. Plagiarism will result in 0% and referral for disciplinary action in line with the Plagiarism Policy of CPUT. Projects will also be checked through ChatGPT-generated content and similarity tools.
6. Attend your allocated **project presentation/demo session** during Week 12. Each group will be required to demonstrate their system within 7–10 minutes.
7. Failure to submit all required deliverables or attend the demo session will result in a **50% marks deduction**.

Project Overview

Software design is the structured process of planning, organizing, and implementing software systems. It involves breaking down a problem into smaller modules, identifying how data flows through the system, and ensuring that the program is both maintainable and extendable. A well-designed system separates responsibilities into classes, applies algorithms and data structures appropriately, and follows good programming practices such as modularity, reusability, and clear documentation.

In this project, your task is to apply software design principles to develop a **working prototype of a real-world system utility or security tool**. The focus is not just on writing code, but on designing a solution, documenting the approach, and evaluating the results. Each project will require you to apply multiple Java programming concepts from the syllabus, such as object-oriented programming, strings and regular expressions, recursion, file handling, file streams and object serialization, algorithms and Big O notations, generic collections, multithreading, etc. You will work in teams of 5–8 students over a period of **6 weeks** to design, implement, test, and present your project.

General Project Requirements

Regardless of your allocated topic, all projects should include the following elements:

1: Problem Definition

- State the real-world problem your tool solves.
- Explain why it matters.

2: System Design

- Draw a UML class diagram of the system.
- Specify your classes and methods.
- Identify data flow and relationships between entities.

3: Core Features (Minimum Required)

- Input/Output: Read files, user input, or network data.
- Processing Logic: Apply rules, algorithms, or encryption methods.
- Data Structures: Use collections or custom generics for storage.
- Recursion/Algorithms: Handle traversal or searching/sorting.
- Logging/Reporting: Keep records of results, flagged data, or errors.
- Concurrency (Optional): Use threads for parallel operations.

4: Implementation

- Organize code into separate classes.
- Apply Java best practices (naming, comments, error handling).
- Write a CLI menu to test your idea & simple GUI interface for running your application.

5: Testing & Evaluation

- Create your unique sandbox, sample input files, or datasets.
- Test with normal and edge cases.

- Evaluate correctness, efficiency, and ease of use.

6: Documentation & Demo

- Write a system design document (explain the problem, class diagrams, design decisions, data structure and algorithms, advanced features, and testing strategy, reflections on GA2 and GA7, team collaboration process).
- Prepare a 7–10 minute group demo to present your project.

Deliverables: By submission, your group must provide:

1. **Source Code**: organized within a zipped java project.
2. **ReadMe**: instructions, requirements, and user guide.
3. **System Design Document**: 5–10 pages.
4. **Graduate Attribute Reflection:** a short section in your report describing how your project demonstrates GA2 and GA7.
    - **GA2** (Application of Scientific and Engineering Knowledge): Show how you applied Java and software engineering concepts (OOP, recursion, collections, algorithms, file handling, etc.) to solve a real-world problem.
    - **GA7** (Sustainability and Impact of Engineering Activity): Discuss how your project promotes efficient use of resources and its impact on sustainability in software development.
5. **Demo Presentation:** 7–10 minutes in class.

Group Project Topics

No	Project Topic	Brief Description	Groups
1	Mini Antivirus	Build a tool that scans folders for suspicious files by using rules or extensions. It should quarantine flagged files and keep a log of all activity.	16, 40
2	Duplicate File Finder	Create a program that searches a folder and finds files with identical content. Use hashing to compare files and display duplicates clearly.	26, 34
3	File Integrity Monitor	Design a tool that records file hashes and checks if files are modified later. This helps detect tampering or unwanted changes.	14, 35
4	Data Backup Scheduler	Build a system that allows users to select folders to back up automatically at set intervals. It should keep multiple backup versions for recovery.	22, 32
5	Log Analyzer for Intrusion Detection	Parse system or server log files and search for unusual activity like repeated login failures. Summarize and report potential security issues.	33, 36
6	Secure Notes Application	Design an app where users can write and save personal notes. All notes are encrypted so only the user with the correct password can read them.	12, 17
7	USB Device Activity Monitor	Create a program that tracks when USB devices are plugged in or removed. It should log the device details and time of connection.	10, 37
8	Simple Task Scheduler	Write a program that runs tasks automatically at scheduled times. Users should be able to add or remove scheduled tasks easily.	15, 28
9	Contact Management System	Build an app to store names, phone numbers, and emails. It should allow searching, editing, and deleting contacts.	30, 39
10	Text Analyzer Tool	Create a program that analyzes text for word counts, spell check common words, or sentiment. It should produce a summary of key insights.	23, 24
11	Personal Finance Tracker	Develop a tool where users enter expenses and incomes. The system categorizes spending and gives simple charts to show financial habits.	6, 19
12	Taxi Route Optimizer	Create a program that finds the shortest route for buses or taxis using	7, 9

		sample city data. It should suggest the most efficient path to save fuel and time.	
13	Marks Administration System	Develop a system that allows lecturers to capture, import, and manage student marks, with automatic calculation of totals and final grades. The system should also support CSV uploads/exports, flag missing entries, and generate summary reports. Ensure role-based access (lecturer, HoD).	3, 21
14	Secure Chat Application	Develop a chat system where messages are encrypted before being sent. The receiver should decrypt the messages to read them.	27, 38
15	Custom File Encryption Tool	Build your own encryption algorithm to lock and unlock files. Users should be able to select a file, encrypt it, and decrypt it back.	1, 4
16	Password Vault	Create a secure vault to store passwords. All passwords should be encrypted so only the correct master key can unlock them.	20, 25
17	Code Plagiarism Checker	Develop a program that compares multiple student code files to detect similarity. Use string matching, tokenization, or hashing to highlight overlaps.	2, 8
18	Smart File Organizer	Create a tool that automatically organizes files into folders (e.g., documents, images, videos) based on file type and content. It should also log all changes made.	13, 29
19	Simple Version Control System	Build a mini Git-like tool that tracks file changes and allows users to commit, view history, and roll back to previous versions. This helps students understand the basics of version control.	11, 31
20	Simple Payroll System	Create a program to calculate employee salaries based on hours worked, overtime, and deductions. It should generate payslips for each employee.	5, 18

Assessment Rubric

Category	Criteria	Marks
Problem Definition & Requirements	Clear statement of the real-world problem; correct identification of user needs	10
System Design & Modularity	Use of design principles (modularity, abstraction, multiplicity, etc.); appropriate class diagrams/architecture; logical decomposition of system.	15
Data Structures & Algorithms	Correct choice of data structures (collections, custom classes); efficient use of recursion/search/sort where applicable; awareness of complexity (Big O).	15
Implementation Quality	Code correctness, readability, comments; proper use of OOP; consistent error handling and validation	20
Concurrency & Advanced Features	Effective use of multithreading where needed; integration of advanced Java concepts (generics, streams, serialization) to enhance design.	10
Testing & Evaluation	Systematic testing with normal and edge cases; evidence of verification and validation; discussion of performance/limitations.	10
Documentation & Presentation	Clear README and design document; diagrams and explanations of design decisions; project demo (7–10 minutes).	10
Graduate Attributes Reflection	Explicit demonstration of GA2 (application of knowledge) and GA7 (sustainability and efficiency).	5
Team Collaboration & Process	Project participation with evidence, etc. version control/code sharing (if used).	5
		100%